CSE310: Compiler Sessional, January 2026 Term

ICG Online Assignment (A1)

Initialization during Declaration

Time: 40 Mins Total Marks: 10

Problem

In this online, you are required to extend the grammar to support **initialization during variable declaration**. Thus, your program should support the following syntax.

```
int a=10, b=10, c;
```

Task

1. Extend the current grammar to support **initialization during variable declaration**. Initialization can be for both global and local variable declarations. Like the offline, you need to only consider integer type variables. There will be no syntax or semantic errors in the given input program.
2. Update your ICG code to generate assembly code for this new rule/syntax. The assembly code output by your program must correctly be assembled with 'fasm' . Subsequently, the binary produced by 'fasm' for against your assembly code must produce the expected output in the input program.

Mark Distribution

- Extending the grammar to support initialization during variable declaration. (4)
- Code generation for global variable initializations. (3)
- Code generation for local variable initialization. (3)

Find 3 (three) sample input-output in the next pages.

Sample Input-Output

- Program with global variable initialization

Input Program

```
1 int a=10, b=10, c;
2 int main(){
3     c=a+b;
4     println(a);
5     println(b);
6     println(c);
7     return 0;
8 }
9
```

Expected Output (from the generated binary by fasm)

```
10
10
20
```

Generated Sample Assembly (print library omitted)

```
format ELF executable 3
entry main
segment readable writeable
      a dd 1 DUP (10)
      b dd 1 DUP (10)
      c dd 1 DUP (0)
segment readable executable
main:
      PUSH EBP
      MOV EBP, ESP
      MOV EAX, [b]        ; Line 3
      MOV EDX, EAX
      MOV EAX, [a]        ; Line 3
      ADD EAX, EDX
      MOV [c], EAX
.L1:
      MOV EAX, [a]        ; Line 4
      CALL print_number
.L2:
      MOV EAX, [b]        ; Line 5
      CALL print_number
.L3:
      MOV EAX, [c]        ; Line 6
      CALL print number
.L4:
      MOV EAX, 0        ; Line 7
      JMP .L5
.L5:
      POP EBP
      MOV EAX,1
      XOR EBX, EBX
      INT 0x80
      POP EBP
      RET
```

- Program with local variable initialization

Input Program

```
1 int main(){
2     int a=10, b=10, c;
3     c=a+b;
4     println(a);
5     println(b);
6     println(c);
7     return 0;
8 }
9
```

Expected Output (from the generated binary by fasm)

```
10
10
20
```

Generated Sample Assembly (print library omitted)

```
format ELF executable 3
entry main
segment readable writeable
segment readable executable
main:
        PUSH EBP
        MOV EBP, ESP
        SUB ESP, 4
        MOV EAX, 10         ; Line 2
        MOV [EBP-4], EAX
        SUB ESP, 4
        MOV EAX, 10         ; Line 2
        MOV [EBP-8], EAX
        SUB ESP, 4
.L1:
        MOV EAX, [EBP-8]        ; Line 3
        MOV EDX, EAX
        MOV EAX, [EBP-4]        ; Line 3
        ADD EAX, EDX
        PUSH EAX
        POP EAX         ; Line 3
        MOV [EBP-12], EAX
.L2:
        MOV EAX, [EBP-4]        ; Line 4
        CALL print_number
.L3:
        MOV EAX, [EBP-8]        ; Line 5
        CALL print_number
.L4:
        MOV EAX, [EBP-12]        ; Line 6
        CALL print_number
.L5:
        MOV EAX, 0        ; Line 7
        JMP .L6
.L6:
        ADD ESP, 12
        POP EBP
        MOV EAX,1
        XOR EBX, EBX
        INT 0x80
        POP EBP
        RET
```

- Program with both global and local variable initialization

Input Program

```
1 int a=10, b=20, c;
2 int main(){
3     int x=30, y=40, z;
4     c=x+y;
5     z=a+b+c;
6     println(c);
7     println(z);
8     return 0;
9 }
10
```

Expected Output (from the generated binary by fasm)

```
70
100
```

Generated Sample Assembly (print library omitted)

```
format ELF executable 3
entry main
segment readable writeable
      a dd 1 DUP (10)
      b dd 1 DUP (20)
      c dd 1 DUP (0)
segment readable executable
main:
      PUSH EBP
      MOV EBP, ESP
      SUB ESP, 4
      MOV EAX, 30        ; Line 3
      MOV [EBP-4], EAX
      SUB ESP, 4
      MOV EAX, 40        ; Line 3
      MOV [EBP-8], EAX
      SUB ESP, 4
.L1:
      MOV EAX, [EBP-8]        ; Line 4
      MOV EDX, EAX
      MOV EAX, [EBP-4]        ; Line 4
      ADD EAX, EDX
      PUSH EAX
      POP EAX        ; Line 4
      MOV [c], EAX
.L2:
      MOV EAX, [b]        ; Line 5
      MOV EDX, EAX
      MOV EAX, [a]        ; Line 5
      ADD EAX, EDX
      PUSH EAX
      MOV EAX, [c]        ; Line 5
      MOV EDX, EAX
      POP EAX        ; Line 5
      ADD EAX, EDX
      PUSH EAX
      POP EAX        ; Line 5
      MOV [EBP-12], EAX
.L3:
      MOV EAX, [c]        ; Line 6
      CALL print_number
.L4:
      MOV EAX, [EBP-12]        ; Line 7
      CALL print_number
.L5:
      MOV EAX, 0        ; Line 8
      JMP .L6
.L6:
      ADD ESP, 12
      POP EBP
      MOV EAX,1
      XOR EBX, EBX
      INT 0x80
      POP EBP
      RET
```